M365 Mail Client — Build Spec

Oct 11, 2026 · @Shane

Purpose and how to use this spec

Build a native desktop client for Microsoft 365 mail, calendar and contacts on Linux, Windows and macOS, using Microsoft Graph only. Linux is the priority platform. It should feel familiar to Outlook users, but it is not an Outlook clone and does not aim for feature parity. Every function is also available to AI agents through an MCP server, the CLI and a JSON control channel.

Why now: Microsoft began turning off EWS in Exchange Online on 1 October 2026. Thunderbird's native Exchange support is EWS-based and mail-only, with Graph, calendar and contacts still on its roadmap. Evolution has a Graph backend but a GNOME-style experience. A fast, Graph-native client with strong calendaring fills a real gap on Linux. On Windows and macOS the draw is speed, offline reliability and full agent control.

Instructions for the implementing agent:

1. Read the whole spec before writing code. Treat the Requirements sections as the contract and everything else as guidance.
2. Verify every Graph endpoint, scope, header and limit against current Microsoft Learn documentation before using it. API details in this spec were written from memory and may be out of date.
3. Keep a `SOURCES.md` listing every public document consulted (see Clean room rules).
4. Work milestone by milestone. Do not start a milestone until the previous one meets its acceptance criteria.
5. When you hit an item under Open decisions, stop and ask the owner rather than guessing.
6. Never trade data safety for speed. A missing feature is acceptable; a lost email or a dropped meeting is not.

Throughout, "the client" means this application. Its product name is still to be decided and must not contain "Outlook" or any Microsoft trademark.

Goals, non-goals and target users

Version 1.0 succeeds when a Linux user on a Microsoft 365 work account can live in the client all day for mail and calendar without opening Outlook on the web.

Goals:

- Mail, calendar and contacts for Microsoft 365 work and school accounts via Graph.
- Calendaring good enough for meeting-heavy office work: invites, responses, recurrence, free/busy, Teams links, shared calendars.
- Fast with large mailboxes: 100,000+ messages and 20 GB+ per account.
- Full offline reading, search and drafting, with a reliable outbox.
- Outlook-familiar three-pane layout and keyboard shortcuts.
- Native builds for Linux (priority), Windows and macOS from one codebase, each integrated properly with its desktop: notifications, OS secret store, mailto handler, file associations.
- Full agent support: every user-facing function is a command reachable over MCP, the CLI and a JSON control channel, headless or attached to the running app.
- Multiple accounts.

Non-goals for 1.0:

- Feature parity with classic or new Outlook.
- On-premises Exchange (MAPI, EWS). Thunderbird and Evolution already cover EWS.
- COM, VSTO or Office web add-ins, VBA and macros. Agents and MCP replace add-ins as the extension model.
- Public folders, Outlook journal, notes.
- Server-side push notifications (they need a public webhook endpoint).

Stretch goals after 1.0: IMAP/SMTP and CalDAV/CardDAV accounts, personal Microsoft accounts (outlook.com), Microsoft To Do tasks, S/MIME.

Target users: Linux-using developers, engineers and admins in Microsoft 365 organisations, plus small businesses on M365 that run Linux desktops. Secondary: Windows and Mac users who want a faster client or want agents to work their mail and calendar.

Clean room rules

The client is built only from public documentation and open standards. No Microsoft code, private protocol or brand asset may enter the codebase.

Allowed sources:

- Microsoft Graph documentation on Microsoft Learn, including the Graph OpenAPI descriptions.
- Microsoft Identity platform (Entra ID) documentation for OAuth.
- Microsoft Open Specifications, for example MS-PST for PST import.

- IETF RFCs: 5322 and 2045–2049 (MIME), 5545 (iCalendar), 5546 (iTIP), 6350 (vCard), 6749 and 7636 (OAuth, PKCE), 8252 (native app OAuth).
- Unicode CLDR data, including the Windows-to-IANA time zone mapping.
- Open source libraries with compatible licences, used as dependencies.

Forbidden:

- Decompiling, disassembling or inspecting Outlook, OWA or Outlook mobile binaries or bundles.
- Capturing or replaying traffic from Microsoft's own clients (OWA service endpoints, the Outlook mobile sync service, MAPI).
- Using undocumented Graph endpoints or beta endpoints in production code paths. Beta endpoints may be used only behind a feature flag and noted in SOURCES.md.
- Copying Outlook icons, illustrations, sounds, names or UI text verbatim.
- Using "Outlook", "Microsoft" or "Office" in the product name or logo.

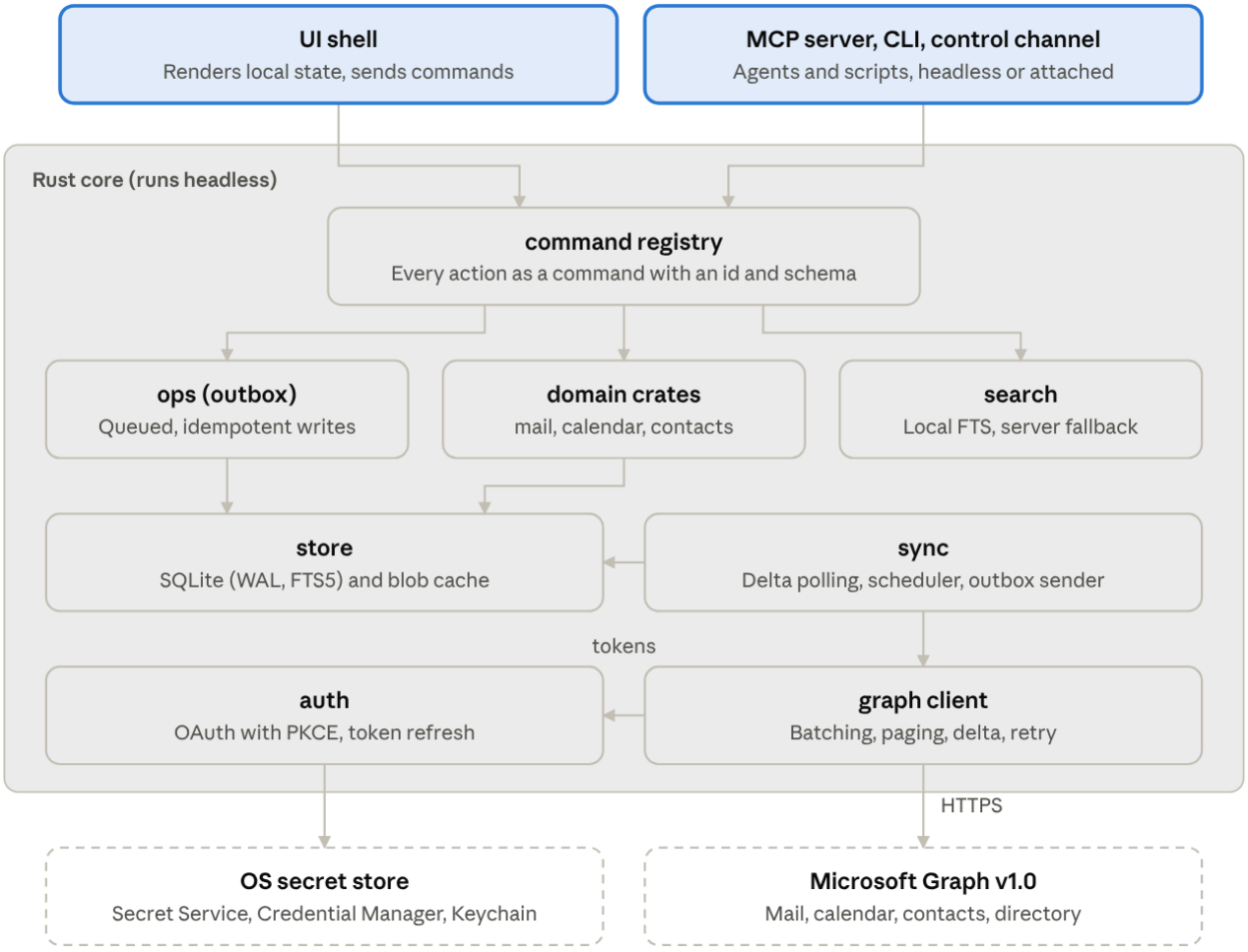
Familiar layout and keyboard shortcuts are fine; they are functional conventions. Icons must come from an open icon set (for example Adwaita or Lucide) or be original.

Provenance: maintain `SOURCES.md` with each external document consulted, its URL and what it informed. Code comments for non-obvious protocol behaviour should cite the relevant doc.

Tech stack and architecture

A Rust core owns all data, sync and network access; a thin UI layer only renders state and sends commands. This keeps the hard, correctness-critical code testable without a UI and leaves the UI swappable.

The UI and agents share one command registry; only the core talks to Graph



client architecture · UI, core crates, external services

The UI and agents dispatch through the same command registry and read only from the local store; writes queue in the outbox, and the sync crate sends them and pulls changes through the Graph client.

Default stack (the UI toolkit is an open decision, see the last section):

Layer	Choice	Notes
Core language	Rust, stable toolchain	Cargo workspace, one crate per component below
Async runtime	tokio	One runtime in the core process
HTTP	reqwest with rustls	HTTP/2, gzip, connection reuse
Local store	SQLite via rusqlite, WAL mode	FTS5 for search; one database per account

Layer	Choice	Notes
Body and attachment cache	Content-addressed files on disk	Per-platform data directory (directories crate); bodies optionally compressed
Secrets	OS secret store via the keyring crate	Secret Service (Linux), Credential Manager (Windows), Keychain (macOS); refresh tokens never touch disk in plain text
UI shell	Tauri 2 with a TypeScript frontend (Svelte)	WebKitGTK, WebView2 and WKWebView give one HTML engine per platform for mail rendering; alternative is egui, as ArtCraft uses
HTML mail rendering	Sanitised HTML in an isolated, script-free view	See Security requirements
Parsing	mail-parser (MIME), icalendar, a vCard crate	Fuzz all parsers

Core components (crates):

- `auth` – OAuth flows, token refresh, account registry.
- `graph` – typed Graph client: batching, paging, delta, throttling and retry.
- `store` – SQLite schema, migrations, FTS index, blob cache.
- `sync` – per-account sync scheduler, delta engines per resource, conflict handling.
- `ops` – durable operation queue (outbox): every user change is a queued, idempotent operation.
- `mail`, `calendar`, `contacts` – domain logic: threading, recurrence expansion, time zone conversion, iTIP handling.
- `search` – local FTS plus server search fallback.
- `pst` – read-only PST importer.
- `commands` – the command registry: every action, query and UI navigation step as a typed command with a stable id and JSON schema.
- `control` – JSON-lines control channel on a local socket, so external tools can drive a running app.
- `mcp` – MCP server over stdio, headless or bridged to a running app through `control`.
- `cli` – command-line binary: run any command, export, import, start the MCP server.

- `platform` – per-OS integration: notifications, secret store, tray, protocol and file handlers.

Rules:

- The UI, keyboard shortcuts, CLI, control channel and MCP server all dispatch through the command registry. No feature may exist only in the UI.
- The UI never calls Graph directly. All reads come from the local store; all writes go through `ops`.
- Every write is applied optimistically to the local store, queued, sent, then confirmed or rolled back with a visible error.
- The core must run headless for tests, the CLI and the headless MCP server.
- Platform-specific code lives only in `platform` and the UI shell; everything else compiles unchanged on Linux, Windows and macOS.

Authentication and app registration

The client is an Entra ID public client app using the authorization code flow with PKCE and a loopback redirect, per RFC 8252. Tenant policy, not code, is the most likely reason the client fails to work at a given organisation, so setup errors must be explained in plain language.

Requirements:

- Register one multi-tenant app ("Accounts in any organizational directory") as a public client. Redirect URI: `http://localhost` (loopback, any port).
- Sign-in opens the system default browser (through the xdg-desktop-portal OpenURI interface on Linux). Never embed a login webview.
- Device code flow as a fallback for headless or broken-browser setups.
- Bring-your-own client ID: an advanced setting lets an organisation use its own app registration and tenant ID. Many tenants block unverified third-party apps, and this is the workaround.
- Request the smallest scope set that works. Starting point, to verify against docs: `offline_access`, `User.Read`, `Mail.ReadWrite`, `Mail.Send`, `Mail.ReadWrite.Shared`, `Mail.Send.Shared`, `Calendars.ReadWrite`, `Calendars.ReadWrite.Shared`, `Contacts.ReadWrite`, `MailboxSettings.ReadWrite`, `People.Read`, `User.ReadBasic.All`, `Place.Read.All`.
- Flag any scope that needs admin consent in the UI and in the docs.
- Store refresh tokens in the OS secret store only. Access tokens live in memory. Never log tokens, auth codes or full Graph URLs containing them.

- Refresh proactively before expiry; on `invalid_grant` or a conditional access failure, mark the account "needs sign-in" and keep everything else working offline.
- Handle claims challenges (Continuous Access Evaluation): on a 401 with a claims challenge, re-run interactive sign-in with those claims.

Error messages the UI must cover in plain language: admin consent required, app blocked by tenant, conditional access or device compliance required, MFA required, account disabled, wrong account type (personal account in 1.0).

Publisher verification: before public release, complete Microsoft publisher verification for the app registration. This reduces consent warnings and the chance of being blocked.

Sync engine and local storage

Sync uses Graph delta queries with polling, a durable outbox for writes, and a local store that is the single source of truth for the UI. Correctness beats freshness: a sync that is a minute late is fine; a sync that loses or duplicates data is a release blocker.

IDs and headers:

- Send `Prefer: IdType="ImmutableId"` on every mail and event request so IDs survive moves between folders.
- Send `Prefer: outlook.timezone` only where useful; store all times in UTC plus the original time zone.
- Store Graph `changeKey` / `@odata.etag` per item and use `If-Match` on updates.

Read sync:

1. Folder tree via `mailFolders/delta`, including child folders and well-known folder names.
2. Messages per folder via `messages/delta`. Initial sync fetches headers and metadata newest first, so the inbox is usable within seconds; bodies download on open and in the background for the last N days (default 90, configurable).
3. Events via `calendarView/delta` over a rolling window (default: 1 year back, 2 years forward), plus series masters for recurrence.
4. Contacts via `contacts/delta` and `contactFolders`.
5. Mailbox settings (time zone, working hours, automatic replies) and master categories on a slow timer.

Scheduling:

What	Interval	Notes
Focused folder (open in UI)	30–60 s	Also on window focus and on F9
Inbox and favourites	2 min	
Other folders	10–15 min	Staggered
Calendar window	2 min	
Contacts, settings, categories	30 min	

After a network change, resume, or wake from sleep, sync the focused folder and inbox immediately.

Robustness requirements:

- Persist the delta link after each fully applied page, inside the same SQLite transaction as the page's changes. A crash mid-sync must resume without loss or duplication.
- On an expired or invalid delta token (e.g. HTTP 410 or `syncStateNotFound`), run a full resync of that resource that reconciles against local state; never wipe and refetch blindly.
- Honour `429` and `503` with `Retry-After`; otherwise exponential backoff with jitter. Keep per-mailbox concurrency low (start with 4 in-flight requests) and use JSON `$batch` (up to 20 requests) where it reduces calls.
- Initial sync of a large mailbox must be resumable, show progress, and stay under Graph throttling limits.
- Deletions arrive as `@removed` entries; apply them, then garbage-collect cached bodies and attachments.

Write path (outbox):

- Every user action (flag, move, delete, mark read, send, RSVP, event edit) becomes a row in an `ops` table with a client-generated idempotency key.
- Ops for the same item run in order; independent ops run in parallel.
- A failed op retries with backoff; a permanent failure rolls back the local change and shows a notification with the reason.
- Conflict rule: if an `If-Match` update fails because the server changed, refetch, re-apply the user's change if it still makes sense, otherwise keep the server version and tell the user.
- Sends survive restarts: a queued message stays in the Outbox until Graph confirms it.

Storage:

- One SQLite database per account plus a small global database for accounts and settings. Schema migrations are versioned and tested on upgrade from every previous release.
- FTS5 index over subject, sender, recipients, body text and attachment names.
- Cache size limit with LRU eviction of bodies and attachments older than the offline window; metadata is never evicted.
- Optional at-rest encryption of the cache (SQLCipher) is a stretch goal; document that the cache relies on disk encryption by default.

Mail requirements

Priority key: **P0** = required for 1.0, **P1** = should ship in 1.0, **P2** = after 1.0.

Area	Requirement	Priority
Folders	Full folder tree, well-known folders first, unread counts, favourites, create / rename / move / delete folders	P0
List	Virtualised message list that scrolls smoothly at 100k+ items; sort by date, sender, subject, size; group by date	P0
Threading	Conversation view toggle using `conversationId`; per-folder setting	P0
Focused Inbox	Focused / Other tabs from `inferenceClassification`; "always move" overrides	P1
Reading	Reading pane right, bottom or off; open in new window; plain-text and HTML views	P0
Remote content	Blocked by default; per-message and per-sender allow; trusted-senders list	P0
Actions	Read/unread, flag with due date, pin, categories, move, copy, delete, archive, junk / not junk, ignore conversation (P2)	P0
Undo	Undo for move, delete, archive, flag, read state for 10 s	P0

Area	Requirement	Priority
Compose	Rich text editor and plain-text mode; reply, reply all, forward, forward as attachment; inline images; drafts saved to server	P0
Recipients	Autocomplete from recent recipients, contacts and directory; contact groups; resolve against directory before send	P0
Attachments	Add, preview, save, open via portal; uploads over 3 MB via upload session; drag and drop	P0
Send options	Importance, read and delivery receipt requests, delay send	P1
Signatures	Local signatures per account (no public API for roaming signatures)	P0
MailTips	Out-of-office, external recipient and large-audience warnings via `getMailTips`	P1
Rules	View and edit server-side inbox rules via `messageRules`, within what Graph supports	P1
Automatic replies	Set and schedule out-of-office, internal and external text	P0
Search	Instant local search with filters (from, to, has attachment, date, folder); server search fallback for content outside the cache	P0
Search folders	Saved local searches ("unread", "flagged")	P2
Export	Save message as .eml; print via the system print dialog	P1
S/MIME	Read and send signed / encrypted mail using MIME send	P2

The composed HTML must render acceptably in Outlook desktop, OWA and Gmail. Use simple, table-free HTML with inline styles and test against all three.

Calendar requirements

Calendar is the feature most likely to send users back to Outlook, so it gets the most test effort. Every meeting must appear at the right time, in the right state, for every attendee.

Area	Requirement	Priority
Views	Day, work week, week, month, agenda; multiple calendars side by side or overlaid; mini-month navigator	P0
Calendars	Own calendars and calendar groups; colours; show / hide; create and delete own calendars	P0
Events	Create, edit, delete, drag to move and resize; all-day and multi-day; location; body; reminders; show-as (free / tentative / busy / OOF / working elsewhere); private flag; categories	P0
Recurrence	Daily, weekly, monthly (absolute and relative), yearly; end by date, count or never; edit or delete one occurrence, this and following, or whole series	P0
Exceptions	Moved and cancelled occurrences render correctly; exception edits preserved when the series changes where Graph allows	P0
Meetings	Invite attendees (required, optional, resources); send updates only to changed attendees where possible; cancel with a message	P0
Responses	Accept / tentative / decline, with or without a response, with a comment; propose new time	P0
Invites in mail	Meeting requests, updates and cancellations shown in the reading pane with inline RSVP and a conflict preview	P0
Tracking	Attendee response tracking for meetings you organise	P0
Scheduling	Scheduling assistant using `getSchedule` free/busy; suggested times via `findMeetingTimes`	P0 / P1
Rooms	Room list and room finder via `places`	P1
Online meetings	Toggle "Teams meeting" (`isOnlineMeeting` + provider); show join link; one-click join via browser or Teams for Linux PWA	P0
Shared calendars	Open another user's calendar; view shared and delegated calendars; manage sharing permissions on your own	P0 / P1

Area	Requirement	Priority
Time zones	Display in the user's zone; per-event start and end zones; secondary time zone in the day view	P0
Reminders	Desktop notifications with snooze and dismiss, fired locally even when offline; dismiss state synced	P0
External invites	Open .ics files and text/calendar parts from non-Exchange senders (RFC 5546 REQUEST / CANCEL / REPLY)	P1
Working hours	Read from mailbox settings; shade non-working hours	P1

Time zone rules:

- Graph returns Windows time zone names in many places. Map them to IANA zones using the CLDR `windowsZones` table, and ship the mapping as data, not code.
- Expand recurrence in the event's own time zone, then convert. A weekly 9 am meeting stays at 9 am local across DST changes.
- Keep a golden test suite covering DST transitions in Australia/Brisbane (no DST), Australia/Sydney, Europe/London, America/New_York and a half-hour zone (Asia/Kolkata).

Prefer server-side expansion (`calendarView`) for display, and use local expansion only for offline ranges and reminders. Where both exist, a mismatch is a bug to log, and the server wins.

Contacts, people and directory

Personal contacts sync fully offline; the organisation directory is searched live and cached only for recently used people.

Area	Requirement	Priority
Personal contacts	List, search, create, edit, delete; contact folders; photos	P0
Contact groups	Use existing groups when addressing mail	P1
Directory	Search the organisation directory (`users`, People API) for autocomplete and lookup; show title, department, phone, manager	P0

Area	Requirement	Priority
Distribution lists	Resolve and expand groups when composing; show membership	P1
Contact card	Hover card for any sender: details, recent mail, upcoming meetings together	P1
Import / export	vCard import and export	P1
Presence	Teams presence dots	P2 (needs extra scopes)

Shared mailboxes, delegation and PST import

Shared mailboxes are the most common work need after the basics, and Graph will not discover them automatically, so the user adds them by address.

Shared and delegated mailboxes:

- Add a shared mailbox by email address; it appears as its own folder tree under the account and syncs like a mailbox, using `/users/{address}/...` paths and the `.Shared` scopes. **P0**
- Send as and send on behalf of a shared mailbox or delegator, chosen from a From picker. Surface the server's error clearly when the user lacks the permission. **P0**
- Sent items from a shared mailbox are saved to the shared mailbox's Sent Items where the tenant is configured for it; document the behaviour. **P1**
- Delegate calendar access: view and respond to meetings on a manager's calendar where permissions allow. **P1**
- Auto-discovery of mailboxes the user has full access to is not available through Graph; do not attempt workarounds.

PST import (read-only, **P1**):

- Parse PST and OST files per MS-PST (Unicode format; ANSI format best-effort), using or adapting an existing open-source parser where the licence allows.
- Import into a local-only "Archives" store that is searchable with the same UI. Do not upload to the server in 1.0.
- Preserve folder structure, dates, read state, flags, attachments and calendar items where present.
- Handle 50 GB files without loading them into memory; show progress and allow cancellation.
- Never modify the source file.

UI, keyboard and Linux integration

An Outlook user should find their way around in the first minute, but the look is the client's own: clean, flat, native-feeling on GNOME, KDE, Windows 11 and macOS.

Layout:

- Left navigation rail switching Mail, Calendar, People.
- Mail: folder pane, message list, reading pane (right, bottom or off); collapsible folder pane.
- A compact command bar with the common actions, plus an overflow menu. No ribbon.
- Command palette (Ctrl+K) listing every action with its shortcut.
- Light, dark and follow-system themes; respect the system accent colour and font scaling.
- Fully usable at 1280×720 and on HiDPI; keyboard-only operation; screen reader labels on all controls (AT-SPI, UI Automation, macOS Accessibility).

Performance targets on a mid-range laptop:

Measure	Target
Cold start to usable inbox (synced account)	< 1.5 s
Switch folder	< 100 ms
Open cached message	< 100 ms
Local search results	< 200 ms on 100k messages
Idle memory	< 300 MB
Idle CPU	~0% between syncs

Default keyboard shortcuts (Outlook-compatible, all rebindable):

Action	Shortcut
New mail / new item	Ctrl+N
Reply	Ctrl+R
Reply all	Ctrl+Shift+R
Forward	Ctrl+F
Send	Ctrl+Enter

Action	Shortcut
Mark read / unread	Ctrl+Q / Ctrl+U
Delete	Delete
Move to folder	Ctrl+Shift+V
Search	Ctrl+E or F3
Sync now	F9
Go to Mail / Calendar / People	Ctrl+1 / Ctrl+2 / Ctrl+3
Next / previous message	Ctrl+. / Ctrl+,
New meeting	Ctrl+Shift+Q
Address book	Ctrl+Shift+B
Command palette	Ctrl+K

On macOS, Cmd replaces Ctrl throughout. Note that Ctrl+F means Forward, as in Outlook; find-in-message is Ctrl+Shift+F or F4. Offer an optional Gmail-style single-key scheme as a setting (P2).

Platform integration:

- **All platforms:** desktop notifications for new mail (focused inbox only by default) and reminders, with actions (mark read, delete, snooze); background running with autostart; single instance, so `mailto:` links focus the running window; registered as the `mailto:` handler and for `.eml` and `.ics` files; system proxy settings respected.
- **Linux:** Wayland-first with X11 support; tray via StatusNotifierItem; xdg-desktop-portal for file chooser, open URI, print and background permission; D-Bus activation; XDG base directories.
- **Windows 10 and 11:** native toast notifications with actions; system tray icon; jump list (new mail, new meeting); registered as a default mail app in Settings; per-monitor DPI awareness; dark mode and accent colour from the system.
- **macOS 13 and later:** menu bar with standard app menus and Cmd shortcuts; Dock badge for unread count; UserNotifications with actions; registered as a default mail reader option; universal binary (Apple silicon and Intel); respects system appearance and accent colour.

Agent and MCP support

Every function a user can perform is a command an agent can run. This follows ArtCraft's model: one command registry sits behind the UI, keyboard shortcuts, the CLI, a JSON-lines control channel and an MCP server, so nothing is reachable only by clicking. Below, `<app>-cli` stands for the CLI binary until the product is named.

Command registry:

- Every menu item, button, shortcut, drag action and setting change maps to a command with a stable dotted id (for example `mail.reply_all`, `calendar.event.create`), a JSON Schema for its arguments and result, and a description written for agents.
- Queries are commands too: `mail.search`, `calendar.view`, `contacts.lookup`.
- Navigation is exposed as commands (`ui.open_folder`, `ui.select_message`, `ui.open_event`) so an attached agent can show the user what it is doing.
- A CI test enumerates the registry and fails if any command lacks an MCP path, a CLI path, a schema or a test.

Surfaces:

Surface	How it runs	Use
MCP, headless	`<app>-cli mcp` over stdio; opens the local store directly	Agents working while the app is closed; scripts and servers
MCP, attached	`<app>-cli mcp --connect`; bridged to the running app through the control channel	Agents working alongside the user; the UI follows along
Control channel	JSON-lines over a Unix socket (Linux, macOS) or named pipe (Windows), current user only	Scripts and other local tools
CLI	`<app>-cli run <command-id> --json '{…}'`	Shell automation and cron jobs

Only one process owns an account's store at a time. If the app is running, a headless MCP server detects it and forwards through the control channel instead of opening the store.

MCP tools (named tools for common jobs, plus a generic one for everything else):

- **Mail:** `list_folders`, `list_messages`, `get_message`, `get_thread`, `search_mail`, `draft_message`, `update_draft`, `reply`, `reply_all`, `forward` (all create drafts), `send_draft`, `move_messages`, `delete_messages`, `flag_messages`, `mark_read`,

`set_categories`, `get_attachment`, `save_attachment`, `set_automatic_replies`, `list_rules`, `update_rule`.

- **Calendar:** `list_calendars`, `get_events` (date range, any calendar the user can see), `get_event`, `create_event`, `update_event` (scope: occurrence, this and following, or series), `delete_event`, `respond_to_event`, `get_free_busy`, `find_meeting_times`, `find_rooms`.
- **People:** `search_people`, `get_contact`, `create_contact`, `update_contact`, `delete_contact`.
- **App:** `list_accounts`, `sync_now`, `get_sync_status`, `ui_navigate`, `screenshot` (attached mode only).
- **Generic:** `list_commands` and `execute_command`, which run any registry command by id with JSON arguments.
- **Resources:** messages, threads and events addressable by URI (for example `mail://<account>/message/<id>`) so agents can reference them without copying content.

Message bodies are returned as sanitised plain text or Markdown by default, with HTML available on request. Use the official Rust MCP SDK if it meets these needs; verify its current status first.

Agent safety (release blockers):

- **Email is untrusted input.** Tool results mark message content as external data, and tool descriptions tell agents that instructions inside emails are not commands. This is the main defence against prompt injection through incoming mail.
- **Permission tiers** per MCP client: read-only (the default for a newly added client), draft (can create drafts and tentative events but not send or delete), and full. Set in the app's settings or with CLI flags.
- **Consequential commands** need confirmation: sending, deleting, permanent deletion, forwarding outside the organisation, changing rules, automatic replies and calendar sharing. In attached mode the app asks the user; in headless mode an explicit flag (for example `--allow-send`) is required.
- **Rate limits** per agent session, such as a cap on messages sent per hour, to limit damage from a runaway agent.
- **Audit log** of every agent-issued command: client name, redacted arguments, result and time, viewable in the app, with undo where the action allows.
- The control channel is local and per-user only; any optional TCP listener binds to localhost and requires a token. Tokens and raw Graph access are never exposed through MCP.

Deliverables in the repository: a ready `.mcp.json`; `AGENTS.md` with contributor and agent rules (clean room, asset policy, never crash, quality gates); `docs/mcp.md`, `docs/cli.md` and `docs/control-protocol.md`; one-line setup instructions for common MCP clients.

Testing: MCP protocol conformance tests; a test per tool against the fake Graph server; and a nightly agent evaluation in which an LLM agent completes scripted tasks ("accept every meeting from Sam next week", "find the invoice from March and forward it to accounts") against the test tenant, with success rate tracked over time.

Security requirements

An email client renders hostile content all day, so treat every message, attachment and invite as attacker-controlled. Each item below is a release blocker.

Message rendering:

- Sanitise HTML with an allowlist sanitiser (e.g. ammonia) before display. Strip scripts, event handlers, forms, iframes, objects, embeds, meta refresh and `javascript:` / `data:` URLs except inline images.
- Render in an isolated view with JavaScript disabled, a strict Content Security Policy, and no IPC bridge to the app.
- Block remote images, fonts and CSS by default; when allowed, fetch through the core (no cookies, no referrer) rather than from the view.
- Show the real destination of every link on hover; warn when the visible text is a different URL. Open links only via the portal, never inside the app.
- Neutralise CSS that could overlay or spoof app UI.

Attachments and files:

- Open attachments only via the portal or after saving; warn before opening executables (including .exe, .msi, .app and .command files), scripts, `.desktop` files and archives containing them.
- Never auto-open anything.
- Fuzz the MIME, HTML, iCalendar, vCard and PST parsers in CI (cargo-fuzz), and run them with size and depth limits.

Credentials and data:

- Tokens only in the OS secret store; redact tokens, email addresses and subjects from logs by default.
- No telemetry. Crash reports only with explicit opt-in, scrubbed of content.
- The Flatpak manifest requests only the permissions needed: network, Secret Service, notifications, portals. No home directory access.

Supply chain:

- `cargo audit` and `cargo deny` (licences, advisories, duplicate crates) gate every merge.
- Pin and review frontend dependencies; avoid large UI frameworks with deep dependency trees.
- Reproducible release builds; signed releases on every platform (Authenticode on Windows, Developer ID signing and notarisation on macOS).

Before 1.0, run an external review of the rendering and attachment paths.

Testing strategy and quality bar

The core must be provably safe with user data before the UI is polished. Most tests run offline against a fake Graph server; a nightly suite runs against a real tenant.

Test layers:

1. **Unit tests** for every core crate; recurrence, time zone and threading logic get property-based tests (proptest).
2. **Fake Graph server**: an in-process HTTP server implementing the subset of Graph the client uses, including delta links, paging, `$batch`, ETags, throttling, 410 on expired delta tokens, and random 5xx. Fixtures are recorded from a test tenant and anonymised.
3. **Sync chaos tests**: kill the process at random points during sync and outbox processing; inject 429s, timeouts and reordered responses; assert that the local store converges to the server state with no lost or duplicated items.
4. **Live tenant suite (nightly)**: a dedicated Microsoft 365 test tenant with seeded mailboxes (one large: 100k+ messages), a shared mailbox, a room, and a second user for meeting round-trips. Never use a production tenant.
5. **Email rendering corpus**: a few hundred real-world newsletters, receipts and Outlook-generated messages; snapshot-tested for sanitiser output and visual regressions.
6. **Calendar golden tests**: the DST and recurrence cases listed in Calendar requirements, checked against what the server's `calendarView` returns.
7. **UI end-to-end tests** for the main flows: add account, read, reply, search, create meeting, RSVP, add shared mailbox.
8. **Migration tests**: upgrade a database from every previous schema version.

Quality bar for any merge to main:

- All tests on Linux, Windows and macOS runners, Clippy (deny warnings), rustfmt, cargo deny and cargo audit pass.
- New behaviour has tests; bug fixes include a regression test.

- No `unwrap()` / `expect()` in non-test code paths that handle network or user data.

The tenant is an open decision: confirm how a test tenant will be obtained and paid for.

Packaging and distribution

Flatpak is the primary Linux channel; Windows and macOS ship signed native installers with automatic updates.

- **Linux, Flatpak** (Flathub): P0. Minimal permissions as listed under Security.
- **Linux, AppImage**: P1, for users without Flatpak. **.deb and .rpm**: P2.
- **Windows**: signed MSIX or MSI installer, P0; winget manifest, P1. In-app updates with signature checks.
- **macOS**: signed and notarised universal DMG, P0; Homebrew cask, P1. In-app updates with signature checks.
- **CLI and MCP server**: shipped inside every package, plus standalone archives for headless servers, P1.
- Supported baseline: current Ubuntu LTS, Fedora stable, Debian stable and Arch on GNOME and KDE Plasma; Windows 10 22H2 and Windows 11; macOS 13 and later.
- CI on GitHub Actions with Linux, Windows and macOS runners: build, test, lint and package on every pull request; tagged releases publish signed artifacts and a changelog.
- Versioning: SemVer; 0.x until the 1.0 acceptance criteria are met.
- Licence: open decision (see last section).

Milestones and acceptance criteria

Nine milestones take the client from an empty repo to 1.0. Each must meet its criteria, with tests, before the next begins. From M1 on, every milestone also requires that each new command is in the registry with a schema, reachable over MCP and the CLI with a test, and that CI passes on all three platforms.

1. **M0 — Foundations.** Cargo workspace, CI on Linux, Windows and macOS, lint and audit gates, fake Graph server skeleton, command registry, control channel and MCP server skeletons, AGENTS.md, SOURCES.md, and an empty window built for all three platforms.
    - Accept: CI green on all three platforms; the app launches on GNOME, KDE, Windows 11 and macOS; an MCP client connected to the CLI lists the skeleton tools.

2. **M1 — Sign-in and accounts.** OAuth with PKCE via system browser, device code fallback, Secret Service storage, bring-your-own client ID, multiple accounts, plain-language auth errors.
   - Accept: sign in, restart, stay signed in; revoked token shows "needs sign-in" without crashing; BYO client ID works.
3. **M2 — Read-only mail.** Folder and message delta sync, local store, FTS search, list and reading pane, safe HTML rendering, remote content blocking.
   - Accept: 100k-message mailbox syncs resumably under throttling; chaos tests pass; performance targets met for list and search.
4. **M3 — Mail actions and compose.** Outbox, flags, moves, deletes, undo, compose, reply, forward, attachments including upload sessions, drafts, signatures, automatic replies.
   - Accept: sends survive restart and network loss; no duplicate sends in chaos tests; composed mail renders correctly in Outlook desktop, OWA and Gmail.
5. **M4 — Calendar read.** Calendar views, calendar delta sync, time zone mapping, recurrence display, reminders, meeting requests shown in mail.
   - Accept: all golden DST and recurrence tests match server `calendarView`; reminders fire offline.
6. **M5 — Calendar write and meetings.** Create and edit events and series, invitations, RSVPs, cancellations, scheduling assistant, Teams meeting toggle, rooms.
   - Accept: full meeting round-trip between two test users, including a recurring series with an exception and a DST crossing.
7. **M6 — People and sharing.** Contacts, directory search, autocomplete, shared mailboxes, send-as and on-behalf, shared and delegated calendars, categories, rules.
   - Accept: shared mailbox add, read, send-as, and sent items placement verified on the live tenant.
8. **M7 — PST import.** Read-only importer to local Archives store.
   - Accept: a 20 GB PST imports with progress, cancellation and no memory blow-up; contents searchable.
9. **M8 — Hardening and 1.0.** Accessibility pass, external security review, desktop integration polish, docs, Flathub submission, signed and notarised Windows and macOS installers, publisher verification, review of MCP permission tiers and the audit log.
   - Accept: owner uses it as their only mail and calendar client for two weeks without data loss or a blocking bug.

Known Graph gaps, risks and open decisions

A few Outlook features have no clean Graph equivalent; the client should say so honestly rather than fake them. Verify each gap against current documentation during M0, since Graph changes often.

Likely Graph gaps:

Outlook feature	Graph situation (verify)	Plan
Online archive mailbox	Not reliably exposed through Graph	Out of scope; link to OWA
Public folders	Not available	Out of scope
Roaming signatures	No public API	Local signatures
Auto-mapped shared mailboxes	No discovery API	User adds by address
Some rule conditions and actions	`messageRules` covers a subset	Edit supported rules; show others read-only
Push notifications	Webhooks need a public endpoint	Polling with delta
Sweep, Quick Steps	Client-side features in Outlook	Rebuild locally if wanted (P2)

Risks:

Risk	Impact	Mitigation
Tenants block third-party apps	Client unusable at many workplaces	Publisher verification; BYO client ID; clear admin guide
Prompt injection through incoming email	An agent sends, forwards or deletes mail on an attacker's instructions	Email marked as untrusted data; read-only default tier; confirmation for consequential commands; rate limits; audit log
Graph throttling on large mailboxes	Slow first sync	Resumable, newest-first sync; low concurrency; `$batch`
Calendar edge cases	Missed or mistimed meetings, loss of trust	Server expansion as truth; golden tests; live round-trip tests

Risk	Impact	Mitigation
HTML rendering exploits	Account or machine compromise	Sanitiser + isolated script-free view; fuzzing; external review
Three platforms stretch a small team	Linux quality slips	Linux stays priority; platform code isolated in `platform`; CI on all three from M0
AI-generated code quality	Subtle sync and data bugs	Tests before features; chaos suite; human review of `sync`, `ops`, `store`
Graph API changes or deprecations	Breakage	Pin to v1.0; nightly live suite catches drift

Open decisions for the owner:

- [ ] Product name.
- [ ] UI toolkit: Tauri 2 + web frontend (one HTML engine per platform for mail rendering, faster to build) or pure-Rust egui as ArtCraft uses (lighter, but HTML mail would still need an embedded webview).
- [ ] Licence: MIT/Apache-2.0 dual (as ArtCraft) or GPL-3.0.
- [ ] Test tenant: how a Microsoft 365 tenant for the live suite is obtained and funded.
- [ ] Code signing: Apple Developer Program membership and a Windows code signing certificate, and under whose name.
- [ ] Default MCP permission tier for new clients (spec says read-only) and which commands count as consequential.
- [ ] Personal Microsoft accounts (outlook.com) in 1.0 or after.
- [ ] Whether to publish under a company identity for publisher verification.
- [ ] Offline window default (90 days) and cache size default.